

MRC Lecture 2 — Derivation Guide 2

Scaled Dot-Product Attention: Full Derivation

Sai T. Reddy — ETH Zurich / BIIE

1. Motivation

This is the central derivation of Lecture 2 and one of the most important in the course. The **scaled dot-product attention mechanism** is the mathematical core of the transformer architecture. Every major AI system — GPT, Claude, BERT, AlphaFold2, ESM-2, CLIP, CALM — is built on this operation.

In this guide, we derive the attention equation step by step from first principles, trace the dimensions at every stage, and work through a complete numerical example. Every operation in the derivation maps to a tool from Lecture 1: the dot product, matrix multiplication, and softmax with temperature scaling.

In Lecture 6, we will prove that this equation is mathematically identical to the Boltzmann distribution governing molecular binding. For now, we derive it as an AI architecture and build the fluency needed to see that identity when it arrives.

2. Setup: Input Embeddings

Starting point. We have a sequence of n tokens, each represented as a d_{model} -dimensional embedding vector. These embeddings come from a learned embedding layer (for text) or a pre-trained language model (for proteins).

Stack the n embedding vectors as rows of a matrix:

$$X \in \mathbb{R}^{n \times d_{\text{model}}}$$

Row i of X is the embedding vector $x_i \in \mathbb{R}^{d_{\text{model}}}$ for token i .

Example dimensions. For a protein sequence of length $n = 100$ residues encoded by ESM-2 with $d_{\text{model}} = 512$: $X \in \mathbb{R}^{100 \times 512}$.

3. The Three Projections: Q, K, V

Each token's embedding is projected into three separate vector spaces by learned linear transformations.

Definitions. Three learned weight matrices:

$$W_Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}, W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$$

The projections are:

$$Q = X W_Q \in \mathbb{R}^{n \times d_k} \text{ (queries)}$$

$$K = X W_K \in \mathbb{R}^{n \times d_k} \text{ (keys)}$$

$$V = X W_V \in \mathbb{R}^{n \times d_v} \text{ (values)}$$

Interpretation. Each token is asked three questions:

- **Query** q_i (row i of Q): “What am I looking for from other tokens?”
- **Key** k_i (row i of K): “What do I offer to other tokens?”
- **Value** v_i (row i of V): “What information do I carry to transfer if selected?”

The same input embedding x_i produces three different vectors because W_Q , W_K , W_V are different learned matrices. The projections extract different aspects of the token's representation for different roles.

Why separate projections? Without projections, the compatibility score between two tokens would be $x_i \cdot x_j$ — their similarity in the original embedding space, which captures general context but not task-specific relevance. The projections transform the embeddings into spaces optimized for the query-key matching task. The model learns to project tokens into a space where the dot product captures the specific type of relationship relevant to the task.

Dimension note. In the original transformer, $d_k = d_v = d_{\text{model}} / H$ where H is the number of attention heads. For a single head (which we derive first), d_k is a free parameter. Typically $d_k = 64$ in standard transformers.

4. Stage 1: Compute All Pairwise Compatibility Scores

Computation. The compatibility score between query i and key j is the dot product $q_i \cdot k_j$. All pairwise scores are computed simultaneously by matrix multiplication:

$$S = Q K^T \in \mathbb{R}^{n \times n}$$

Derivation of entry (i, j) . From the definition of matrix multiplication (Lecture 1, Derivation Guide 2):

$$S_{ij} = (Q K^T)_{ij} = \sum_{l=1}^{d_k} Q_{il} (K^T)_{lj} = \sum_{l=1}^{d_k} Q_{il} K_{jl} = q_i \cdot k_j$$

Entry (i, j) is the dot product of query i with key j — the compatibility of token i 's query with token j 's key.

Structure of S . The matrix S is $n \times n$:

- Row i contains the compatibility scores of token i 's query with every key in the sequence.
 - Column j contains the compatibility scores of every query with token j 's key.
 - S_{ii} is the self-compatibility of token i (query i matched with its own key).
 - S is generally **not symmetric** because $q_i \cdot k_j \neq q_j \cdot k_i$ (the query and key projections are different).
-

5. Stage 2: Scale and Normalize to Probabilities

Scaling. Divide all scores by $\sqrt{d_k}$:

$$\tilde{S} = \frac{S}{\sqrt{d_k}} = \frac{Q K^T}{\sqrt{d_k}}$$

The scaling factor $\sqrt{d_k}$ serves as the temperature parameter τ (see Derivation Guide 3 of this lecture for the full variance argument). Without scaling, the dot products grow with d_k , causing softmax to saturate.

Normalization. Apply softmax **row-wise** to convert each row of scaled scores into a probability distribution:

$$A = \text{softmax} \left(\frac{Q K^T}{\sqrt{d_k}} \right) \in \mathbb{R}^{n \times n}$$

where softmax is applied independently to each row. Specifically, for row i :

$$A_{ij} = \frac{\exp(\tilde{S}_{ij})}{\sum_{k=1}^n \exp(\tilde{S}_{ik})} = \frac{\exp(q_i \cdot k_j / \sqrt{d_k})}{\sum_{k=1}^n \exp(q_i \cdot k_k / \sqrt{d_k})}$$

Properties of A . Each row of A is a probability distribution:

- $A_{ij} > 0$ for all i, j (positivity, from softmax)
- $\sum_{j=1}^n A_{ij} = 1$ for each row i (normalization, from softmax)

- If $q_i \cdot k_j > q_i \cdot k_l$, then $A_{ij} > A_{il}$ (rank preservation)

Row i of A tells us: for token i , what is the probability distribution over all other tokens? Which tokens does token i attend to, and how strongly?

Connection to Lecture 1. The entry A_{ij} is softmax of the dot product divided by temperature — exactly the temperature-scaled softmax from Derivation Guide 5 of Lecture 1. In the MRC framework, this will be identified as the Boltzmann selection probability (Lecture 6).

6. Stage 3: Aggregate Information

Computation. The attention output is the attention-weighted sum of value vectors:

$$O = AV \in \mathbb{R}^{n \times d_v}$$

Derivation of row i . From the definition of matrix multiplication:

$$o_i = \sum_{j=1}^n A_{ij} v_j$$

The output for token i is a weighted average of all value vectors, where the weights are the attention probabilities A_{ij} .

Interpretation. Token i collects information from all other tokens, but gives more weight to tokens with higher attention scores. If token i attends strongly to token j (A_{ij} is large), then v_j contributes strongly to the output o_i . If token i barely attends to token l (A_{il} is small), then v_l contributes minimally.

In molecular terms (previewing Lectures 6–7): if residue i of the antibody attends strongly to residue j of the antigen, it means residue i 's query (binding features) is highly compatible with residue j 's key (surface features), and the information from residue j (its value vector) is transferred to the output for residue i in proportion to that compatibility.

7. The Complete Attention Equation

Combining all three stages:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Dimension trace:

Quantity	Shape	Computation
X	$n \times d_{\text{model}}$	Input embeddings
W_Q, W_K	$d_{\text{model}} \times d_k$	Learned projection matrices
W_V	$d_{\text{model}} \times d_v$	Learned projection matrix
$Q = XW_Q$	$n \times d_k$	Query projections
$K = XW_K$	$n \times d_k$	Key projections
$V = XW_V$	$n \times d_v$	Value projections
QK^T	$n \times n$	All pairwise compatibility scores
$QK^T / \sqrt{d_k}$	$n \times n$	Temperature-scaled scores
$A = \text{softmax}(\cdot)$	$n \times n$	Attention weight matrix (row-stochastic)
$O = AV$	$n \times d_v$	Attention output

Lecture 1 operations identified:

- QK^T : matrix multiplication (Derivation Guide 2, Lecture 1) — all pairwise dot products
- $/\sqrt{d_k}$: temperature scaling (Derivation Guide 5, Lecture 1) — controls selection sharpness
- softmax: score-to-probability conversion (Derivation Guide 4, Lecture 1) — the convergence equation

- AV : matrix multiplication — weighted information aggregation

8. Worked Example: Full Numerical Computation

Problem. Compute scaled dot-product attention for $n=3$ tokens with $d_k = d_v = 2$.

Given. Input embeddings (already projected — we skip the projection step for clarity):

$$Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}, K = \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}, V = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$$

Token 1 has query $(1, 0)$, key $(1, 1)$, value $(1, 2)$. Token 2 has query $(0, 1)$, key $(0, 1)$, value $(3, 4)$.

Token 3 has query $(1, 1)$, key $(1, 0)$, value $(5, 6)$.

Step 1: Compute $S = Q K^T$.

$$K^T = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}$$

$$\begin{aligned} S_{11} &= (1)(1) + (0)(1) = 1, S_{12} = (1)(0) + (0)(1) = 0, S_{13} = (1)(1) + (0)(0) = 1 \\ S_{21} &= (0)(1) + (1)(1) = 1, S_{22} = (0)(0) + (1)(1) = 1, S_{23} = (0)(1) + (1)(0) = 0 \\ S_{31} &= (1)(1) + (1)(1) = 2, S_{32} = (1)(0) + (1)(1) = 1, S_{33} = (1)(1) + (1)(0) = 1 \end{aligned}$$

$$S = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 2 & 1 & 1 \end{pmatrix}$$

Note: S is not symmetric. $S_{12} = 0$ but $S_{21} = 1$, because query 1 and key 2 interact differently than query 2 and key 1.

Step 2: Scale by $\sqrt{d_k} = \sqrt{2} \approx 1.414$.

$$\tilde{S} = \frac{S}{\sqrt{2}} = \begin{pmatrix} 0.707 & 0 & 0.707 \\ 0.707 & 0.707 & 0 \\ 1.414 & 0.707 & 0.707 \end{pmatrix}$$

Step 3: Apply softmax row-wise.

Row 1: $\tilde{s}_1 = (0.707, 0, 0.707)$.

$$\begin{aligned} \exp(0.707) &= 2.028, \exp(0) = 1.000, \exp(0.707) = 2.028 \\ Z_1 &= 2.028 + 1.000 + 2.028 = 5.056 \end{aligned}$$

$$A_{11} = \frac{2.028}{5.056} = 0.401, A_{12} = \frac{1.000}{5.056} = 0.198, A_{13} = \frac{2.028}{5.056} = 0.401$$

Row 2: $\tilde{s}_2 = (0.707, 0.707, 0)$.

$$\begin{aligned} \exp(0.707) &= 2.028, \exp(0.707) = 2.028, \exp(0) = 1.000 \\ Z_2 &= 2.028 + 2.028 + 1.000 = 5.056 \\ A_{21} &= 0.401, A_{22} = 0.401, A_{23} = 0.198 \end{aligned}$$

Row 3: $\tilde{s}_3 = (1.414, 0.707, 0.707)$.

$$\begin{aligned} \exp(1.414) &= 4.113, \exp(0.707) = 2.028, \exp(0.707) = 2.028 \\ Z_3 &= 4.113 + 2.028 + 2.028 = 8.169 \\ A_{31} &= \frac{4.113}{8.169} = 0.503, A_{32} = \frac{2.028}{8.169} = 0.248, A_{33} = \frac{2.028}{8.169} = 0.248 \end{aligned}$$

$$A = \begin{pmatrix} 0.401 & 0.198 & 0.401 \\ 0.401 & 0.401 & 0.198 \\ 0.503 & 0.248 & 0.248 \end{pmatrix}$$

Verify: Each row sums to 1.000. ✓

Step 4: Compute $O = AV$.

Row 1 of O (output for token 1):

$$O_{11} = 0.401(1) + 0.198(3) + 0.401(5) = 0.401 + 0.594 + 2.005 = 3.000$$

$$O_{12} = 0.401(2) + 0.198(4) + 0.401(6) = 0.802 + 0.792 + 2.406 = 4.000$$

Row 2 of O (output for token 2):

$$O_{21} = 0.401(1) + 0.401(3) + 0.198(5) = 0.401 + 1.203 + 0.990 = 2.594$$

$$O_{22} = 0.401(2) + 0.401(4) + 0.198(6) = 0.802 + 1.604 + 1.188 = 3.594$$

Row 3 of O (output for token 3):

$$O_{31} = 0.503(1) + 0.248(3) + 0.248(5) = 0.503 + 0.744 + 1.240 = 2.487$$

$$O_{32} = 0.503(2) + 0.248(4) + 0.248(6) = 1.006 + 0.992 + 1.488 = 3.486$$

$$O = \begin{pmatrix} 3.000 & 4.000 \\ 2.594 & 3.594 \\ 2.487 & 3.486 \end{pmatrix}$$

Step 5: Interpret.

Token 1 attends equally to tokens 1 and 3 (weight 0.401 each) and weakly to token 2 (weight 0.198). Its output is a near-equal blend of the value vectors of tokens 1 and 3, with a small contribution from token 2.

Token 2 attends equally to tokens 1 and 2 (weight 0.401 each) and weakly to token 3 (weight 0.198). It gathers information primarily from itself and token 1.

Token 3 attends most strongly to token 1 (weight 0.503) and equally to tokens 2 and 3 (weight 0.248 each). Token 1 has the highest compatibility with token 3's query, so token 3's output is dominated by token 1's value vector.

The output for each token is a personalized, weighted summary of the entire sequence — with weights determined by the compatibility between that token's query and all keys.

9. Self-Attention vs. Cross-Attention

The derivation above is **self-attention**: Q , K , and V all come from the same sequence (X). Each token in the sequence attends to all other tokens in the same sequence.

Cross-attention arises when Q comes from one sequence and K , V come from a different sequence.

Setup. Let $X_A \in \mathbb{R}^{n_A \times d_{\text{model}}}$ be the embeddings of sequence A (e.g., a decoder generating an antibody sequence) and $X_B \in \mathbb{R}^{n_B \times d_{\text{model}}}$ be the embeddings of sequence B (e.g., an encoded antigen sequence).

$$Q = X_A W_Q \in \mathbb{R}^{n_A \times d_k}$$

$$K = X_B W_K \in \mathbb{R}^{n_B \times d_k}$$

$$V = X_B W_V \in \mathbb{R}^{n_B \times d_v}$$

The attention computation is identical:

$$\text{CrossAttention} = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

but now $QK^T \in \mathbb{R}^{n_A \times n_B}$ — the score matrix has a different shape because the two sequences may have different lengths. Entry (i, j) is the compatibility of position i in sequence A with position j in sequence B.

In SFMs (Lecture 7). The cross-attention in the SFM decoder implements the convergence equation at the residue-contact level. The cross-attention weight A_{ij} is the probability that position i of the generated sequence corresponds to position j of the partner — exactly the Boltzmann-weighted contact probability.

10. Computational Complexity of Attention

Time complexity. The dominant cost is the matrix multiplication $QK^T \in \mathbb{R}^{n \times n}$, which requires $O(n^2 d_k)$ operations. For self-attention, this is $O(n^2 d_k)$ per layer.

Memory complexity. The attention matrix $A \in \mathbb{R}^{n \times n}$ must be stored, requiring $O(n^2)$ memory.

The quadratic bottleneck. Both time and memory scale quadratically with sequence length n . For $n = 100$ (a short protein): $n^2 = 10,000$ — trivial. For $n = 1,000$ (a typical protein): $n^2 = 10^6$ — manageable. For $n = 10,000$ (a very long protein or a genomic region): $n^2 = 10^8$ — expensive. This quadratic scaling is the primary computational limitation of standard transformers for very long sequences.

For SFMs. In the discriminative SFM (retrieval), the attention is within each encoder (self-attention over the molecular sequence), so n is the sequence length of the molecule being encoded. In the decoder (generation), cross-attention has complexity $O(n_A \cdot n_B \cdot d_k)$ where n_A and n_B are the lengths of the two molecular sequences.

11. The Attention Equation Written as the Boltzmann Distribution

We can now write the attention weight for a single query-key pair in the form that will be identified with the Boltzmann distribution in Lecture 6:

$$A_{ij} = \frac{\exp(q_i \cdot k_j / \tau)}{\sum_{k=1}^n \exp(q_i \cdot k_k / \tau)}$$

where $\tau = \sqrt{d_k}$.

Compare to the Boltzmann binding probability:

$$P(\text{bind } j) = \frac{\exp(-\Delta G_j / k_B T)}{\sum_k \exp(-\Delta G_k / k_B T)}$$

Under the identification $q_i \cdot k_j \leftrightarrow -\Delta G_j / k_B T$ (dot product = dimensionless binding energy) and $\sqrt{d_k} \leftrightarrow k_B T$ (scaling factor = thermal energy), these are the same equation. The attention weight IS the Boltzmann selection probability.

This identification is stated here without proof. The proof — that this is the unique function satisfying five axioms of competitive selection — is the subject of Lecture 6.

12. Summary

Stage	Computation	Shape	Lecture 1 Operation
Project	$Q = X W_Q, K = X W_K,$ $V = X W_V$	$n \times d_k, n \times d_k,$ $n \times d_v$	Matrix multiplication (forward pass)
Score	$S = Q K^T$	$n \times n$	Matrix multiplication (pairwise dot products)
Scale	$\tilde{S} = S / \sqrt{d_k}$	$n \times n$	Temperature scaling
Normalize	$A = \text{softmax}(\tilde{S})$ row-wise	$n \times n$	Softmax (scores $\rightarrow$ probabilities)

Stage	Computation	Shape	Lecture 1 Operation
Aggregate	$O = AV$	$n \times d_v$	Matrix multiplication (weighted sum)

The complete equation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

13. Checkpoint Questions

Q1. In the worked example, $S_{12}=0$ but $S_{21}=1$. Why is the score matrix not symmetric?

Answer: The score matrix is QK^T , where Q and K are computed by different projection matrices ($W_Q \neq W_K$). So the query of token 1 and the key of token 2 give a different dot product than the query of token 2 and the key of token 1. Asymmetry means: “how much token 1 wants information from token 2” is not the same as “how much token 2 wants information from token 1.” This is analogous to molecular recognition where the binding interface is not necessarily symmetric — the CDR loops of the antibody “query” specific epitope features of the antigen, and the reverse direction queries different features. The SFM symmetric loss (Lecture 7) averages both directions precisely because the binding free energy IS symmetric ($\Delta G_{AB} = \Delta G_{BA}$).

Q2. If $d_k=1$ (one-dimensional queries and keys), what does the score matrix QK^T look like?

Answer: $Q \in \mathbb{R}^{n \times 1}$ is a column vector and $K^T \in \mathbb{R}^{1 \times n}$ is a row vector. Their product $QK^T \in \mathbb{R}^{n \times n}$ is a rank-1 matrix (every row is a scalar multiple of K^T). This means all tokens have proportional attention distributions — the model cannot distinguish between tokens’ queries in a meaningful way. This is why $d_k > 1$ is essential: higher-dimensional projections allow richer query-key interactions.

Q3. In the worked example, token 3 attends most strongly to token 1 (weight 0.503). Explain why, using the raw scores.

Answer: Token 3’s query is $q_3 = (1, 1)$. Token 1’s key is $k_1 = (1, 1)$. Their dot product is $1(1) + 1(1) = 2$ — the highest score in row 3 of S . Token 3’s query and token 1’s key point in the same direction (both are $(1, 1)$), so their compatibility is maximal. After scaling by $\sqrt{2}$ and applying softmax, this score difference translates to token 1 receiving about twice the attention weight of tokens 2 and 3.

Q4. What would happen if you applied softmax column-wise instead of row-wise?

Answer: Column-wise softmax would normalize each column of S to a probability distribution. Column j would then represent: “for key j , what fraction of attention does each query give it?” This is the reverse direction — instead of “where does token i attend?” it answers “who attends to token j ?” In standard attention, row-wise softmax is used because each token needs a probability distribution over sources. In the SFM symmetric loss (Lecture 7), both directions are used: row-wise softmax gives the agent-to-target binding probabilities, and column-wise softmax gives the target-to-agent binding probabilities. Both are averaged because $\Delta G_{AB} = \Delta G_{BA}$.

14. Connection to Future Lectures

- **Lecture 2, WB3:** The variance argument for $\sqrt{d_k}$ scaling is derived formally — why the dot product has variance d_k and how scaling restores unit variance.
- **Lecture 2, WB4:** Multi-head attention runs H parallel copies of the attention computation derived here, each on a $d_k = d_{\text{model}} / H$ dimensional subspace.
- **Lecture 3:** The InfoNCE loss computes exactly one row of the attention matrix — the row corresponding to the correct binding partner — and maximizes its probability. The contrastive learning objective IS the attention equation applied to the training scenario.
- **Lecture 6:** The convergence equation proof shows that $\text{softmax}(q \cdot k / \tau)$ is the unique selection probability satisfying five axioms. The attention equation is the Boltzmann distribution — proven, not assumed.

- **Lecture 7:** The SFM architecture prescribes dual encoders producing Q (agent) and K (target), with the cross-attention score $q \cdot k$ identified as the binding energy $-\Delta G / k_B T$. Every component of the attention equation — QKV projections, softmax normalization, temperature, value aggregation — has a physical interpretation derived from the convergence equation.